

Rapport — Patron de conception *Factory Method*

Intention, exemple de référence et application au projet

Projet GenieLogiciels (Spring Boot, JPA)

Patron Factory Method (Méthode de fabrication)

Date 15/04/2026

Résumé. Ce rapport présente le patron *Factory Method* (GoF), puis montre son application dans le projet GenieLogiciels afin de centraliser et fiabiliser la création des différents types d'utilisateurs (Client, Annonceur, FemmeMenage, SousAdmin, SuperAdmin). Il inclut des tableaux de rôles, un diagramme de classes (PlantUML) et une justification des choix.

Sommaire

1. Intention et structure (référence théorique)
2. Exemple illustratif et sources
3. Application au projet GenieLogiciels
4. Livrables
5. Bibliographie / Liens sources

Remarque : les diagrammes sont fournis en PlantUML (code prêt à coller dans un rendu PlantUML).

1. Intention et structure (référence théorique)

1.1 Intention du patron

Intention (GoF). Définir une interface pour créer un objet, mais laisser les sous-classes décider quelle classe concrète instancier. La *Factory Method* permet à une classe de déléguer l'instanciation à des sous-classes.

Intérêts principaux.

- Réduire le couplage en évitant des `new ConcreteProduct()` dispersés.
- Centraliser les règles de création (valeurs par défaut, état initial, etc.).
- Faciliter l'extension (ajout d'un nouveau type de produit sans modifier le code client de façon invasive).

1.2 Participants (GoF)

Participant GoF	Rôle
Product	Type abstrait commun des objets créés
ConcreteProduct	Implémentations concrètes du produit
Creator	Déclare la <i>factory method</i> (contrat de création)
ConcreteCreator	Redéfinit la factory method pour instancier un <code>ConcreteProduct</code>

1.3 Diagramme de structure (PlantUML — intention)

```
@startuml
    FactoryMethod_Structure_Intent
    skinparam classAttributeIconSize 0

    interface Product {
        +operation()
    }

    class ConcreteProductA {
        +operation()
    }
    class ConcreteProductB {
        +operation()
    }

    abstract class Creator {
        {abstract} +factoryMethod() : Product
        +anOperation()
    }
```

```
class ConcreteCreatorA {
    +factoryMethod() : Product
}
class ConcreteCreatorB {
    +factoryMethod() : Product
}

Product <|.. ConcreteProductA
Product <|.. ConcreteProductB

Creator <|-- ConcreteCreatorA
Creator <|-- ConcreteCreatorB

ConcreteCreatorA ..> ConcreteProductA : crée
ConcreteCreatorB ..> ConcreteProductB : crée

note right of Creator
    anOperation() utilise
    factoryMethod() sans
    connaître la classe concrète
end note
@enduml
```

Rendu : coller ce code dans l'outil PlantUML (en ligne ou plugin IDE) pour obtenir l'image du diagramme.

2. Exemple illustratif et sources

2.1 Source principale (ouvrage GoF)

E. Gamma, R. Helm, R. Johnson, J. Vlissides — *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1995. ISBN : **978-0201633612**. Le chapitre *Factory Method* décrit l'intention, la structure et les collaborations entre Creator/ConcreteCreator et Product/ConcreteProduct.

2.2 Source web fiable (FR, diagrammes)

Refactoring Guru — Factory Method (FR) : <https://refactoring.guru/fr/design-patterns/factory-method>

Cette ressource fournit une explication pédagogique avec diagrammes (complément du livre GoF).

2.3 Exemple illustratif (résumé en français)

Exemple classique (GoF). Un framework de documents appelle une méthode de création (`CreateDocument()`) définie dans une classe générique. Les sous-classes redéfinissent cette méthode pour créer le bon document concret (par ex. dessin vs texte), sans que le code générique dépende des classes concrètes.

3. Application au projet GenieLogiciels

3.1 Problème identifié (avant)

Avant l'application du patron (et avant la refactorisation UML), la création des utilisateurs pouvait être réalisée directement via des constructeurs, ce qui dispersait la logique. De plus, une incohérence de modélisation existait : **Utilisateur et Admin étaient deux classes distinctes avec des attributs similaires, sans relation**, entraînant duplication et maintenance difficile.

Problème	Avant / sans patron	Après / avec Factory Method
Couplage	Le client (contrôleur/service) connaît les classes concrètes et les constructeurs.	Le client dépend d'une fabrique abstraite et d'un type abstrait (Utilisateur).
Duplication	Logique de création répartie + duplication de structure Utilisateur/Admin.	Hiérarchie unifiée + création centralisée via factories.
Évolutivité	Ajouter un type implique de modifier plusieurs zones du code.	Ajouter une classe + une factory concrète + un cas dans le provider.

3.2 Correspondance GoF ↔ notre code

Composant (GoF)	Composant dans le projet	Rôle
Product	Utilisateur (abstrait, racine JPA)	Type commun manipulé par le client du patron
ConcreteProduct	Client, Annonceur, FemmeMenage, SousAdmin, SuperAdmin	Sous-types concrets instanciés selon le besoin
Creator	UtilisateurFactory	Déclare <code>creerUtilisateur()</code>
ConcreteCreator	ClientFactory, AnnonceurFactory, FemmeMenageFactory, SousAdminFactory, SuperAdminFactory	Instancient le bon produit concret
Client	UtilisateurRestController + UtilisateurFactoryProvider	Sélection de la factory selon type puis création

3.3 Rôle de chaque composant (projet)

Composant	Rôle
Utilisateur	Product abstrait : super-classe JPA (@Inheritance(JOINED)) avec attributs communs
Client, Annonceur, FemmeMenage, SousAdmin, SuperAdmin	ConcreteProduct : types métier concrets
UtilisateurFactory	Creator abstrait : contrat de création <code>creerUtilisateur()</code>
ClientFactory, ...	ConcreteCreator : création du sous-type correspondant + valeurs par défaut
UtilisateurFactoryProvider	Sélecteur : retourne la factory concrète selon type
UtilisateurRestController	Client : orchestre <code>register</code> , délègue création à la fabrique
UtilisateurRequest	DTO : transporte les champs de saisie + type

3.4 Diagramme de classes (PlantUML — projet)

```
@startuml
    Projet_Factory_Method_Utilisateur
    skinparam classAttributeIconSize 0
    hide empty members
```

```
abstract class Utilisateur <<Entity>> {
    id : Long
    nom : String
    email : String
    mdp : String
    etat : boolean
}

class Client
class Annonceur
class FemmeMenage
class SousAdmin
class SuperAdmin

Utilisateur <|-- Client
Client <|-- Annonceur
Utilisateur <|-- FemmeMenage
Utilisateur <|-- SousAdmin
SousAdmin <|-- SuperAdmin

abstract class UtilisateurFactory <<Creator>> {
    {abstract} +creerUtilisateur() : Utilisateur
}

class ClientFactory
class AnnonceurFactory
class FemmeMenageFactory
class SousAdminFactory
class SuperAdminFactory

UtilisateurFactory <|-- ClientFactory
UtilisateurFactory <|-- AnnonceurFactory
UtilisateurFactory <|-- FemmeMenageFactory
UtilisateurFactory <|-- SousAdminFactory
UtilisateurFactory <|-- SuperAdminFactory

ClientFactory ..> Client : <<create>>
AnnonceurFactory ..> Annonceur : <<create>>
FemmeMenageFactory ..> FemmeMenage : <<create>>
SousAdminFactory ..> SousAdmin : <<create>>
SuperAdminFactory ..> SuperAdmin : <<create>>

class UtilisateurFactoryProvider <<utility>> {
    {static} +getFactory(type: String) : UtilisateurFactory
}

class UtilisateurRestController <<REST>> {
```

```

+register(request: UtilisateurRequest)
}

UtilisateurRestController ..> UtilisateurFactoryProvider
UtilisateurFactoryProvider ..> UtilisateurFactory : retourne
@enduml

```

3.5 Identification des rôles (rédaction)

- **Utilisateur** joue le rôle de **Product** car les fabriques retournent ce type abstrait, permettant au client de manipuler un objet sans connaître sa classe concrète.
- **ClientFactory...** jouent le rôle de **ConcreteCreator** car elles implémentent `creerUtilisateur()` et choisissent la sous-classe instanciée.
- **UtilisateurFactoryProvider** joue le rôle de sélecteur de fabrique (variante « registre ») car il centralise le choix de la factory selon type.
- **UtilisateurRestController** joue le rôle de client du patron : il délègue la création à la fabrique au lieu d'écrire une logique *if/else* avec des constructeurs.

3.6 Problème résolu (synthèse)

Problème	Résolution apportée
Duplication Utilisateur/Admin	Unification via hiérarchie (admins comme sous-types) + création unique via factories
Couplage fort	Le client dépend d'abstractions (Utilisateur, UtilisateurFactory)
Création dispersée	Centralisation dans les classes *Factory
Évolutivité	Nouveau profil : nouvelle sous-classe + nouvelle factory + ajout dans le provider

4. Livrables

1. **Intention & structure** (diagramme PlantUML générique)
2. **Exemple illustratif** + sources (GoF + Refactoring Guru)
3. **Application au projet** : tableaux, diagramme PlantUML du projet, justification des rôles
4. **Conclusion** : bénéfices, limites, perspectives

5. Bibliographie / liens sources

Source	Référence
Gamma et al., <i>Design Patterns</i>	Addison-Wesley, 1995 — ISBN 978-0201633612 (chapitre <i>Factory Method</i>)
Refactoring Guru (FR) — Factory Method	https://refactoring.guru/fr/design-patterns/factory-method
PlantUML (outil)	https://plantuml.com/

Note : Wikipédia peut servir de point de départ, mais ce rapport s'appuie sur des sources de référence (GoF) et une ressource reconnue (Refactoring Guru).